



DELHI TECHNICAL CAMPUS
Greater Noida
Affiliated to GGSIPU and Approved by AICTE & COA



PRACTICAL FILE

SESSION: 2025-26

Data Science using R

Lab (DS-427P)

IV Year, 7th Sem

Submitted to:

Name: Ms. Eirtty Telang

Designation: Assistant Professor

Submitted by:

Name: Arnav Sharma

Enrollment No: 02918002722

Department of Computer Science and Engineering
Delhi Technical Campus, Greater Noida

INDEX

S.NO	EXPERIMENTS	DATE OF EXPERIMENT	DATE OF SUBMISSION	SIGN
1	Installation of R, basic syntax of R.			
2	Describing data, viewing and manipulating data using R.			
3	Visualizing data- tables, charts, plots using R.			
4	Implementing binomial distribution and plotting density and distribution function.			
5	Implementation of probability distribution			
6	Exploratory data analysis- range, mean, variance, median, standard deviation.			
7	Implementation densities of random numbers- distribution in R, making histogram.			
8	Correlations- making of scatter plots, use of scatter plots to investigate relationship between two variables.			
9	Statistical function in R-Statistical inference, contingency tables, chi-square goodness of fit.			
10	Implement linear and logistics regression in R.			

EXPERIMENT 1

AIM- Installation of R, basic syntax of R.

Software Required

- R (latest version)
- RStudio (optional but recommended)

Installation of R

Step 1: Download R

1. Open your browser and go to:
<https://cran.r-project.org/>
2. Click on **Download R for Windows** (or Mac/Linux depending on your system).

Step 2: Install R

1. Click on **Download R x.x.x for Windows**.
2. Run the downloaded .exe file.
3. Follow the installation wizard:
 - Select language
 - Choose installation path
 - Keep default selections
4. Click **Finish**.

Step 3: Install RStudio (Optional)

1. Go to: <https://posit.co/download/rstudio-desktop/>
2. Download **RStudio Desktop (Free)**.
3. Install it normally.

PART B — Basic Syntax of R

1. Assigning Variables

```
x <- 10  
y = 20  
name <- "User"
```

2. Basic Arithmetic Operations

```
a <- 5  
b <- 2  
a + b # Addition  
a - b # Subtraction  
a * b # Multiplication  
a / b # Division  
a ^ b # Power
```

3. Creating Vectors

```
v <- c(1, 2, 3, 4, 5)  
names <- c("A", "B", "C")
```

4. Sequence and Repeat

```
seq(1, 10, by = 2) # Sequence  
rep(5, times = 3)  # Repeat
```

5. Basic Functions

```
sum(v)      # Sum  
mean(v)     # Average  
length(v)   # Number of elements
```

6. Printing Output

```
print("Hello R")  
print(v)
```

7. Data Frame Creation

```
df <- data.frame(  
  ID = c(1, 2, 3),  
  Name = c("A", "B", "C"),  
  Marks = c(90, 85, 92)  
)
```

8. Plotting in R

```
plot(v, type="o", main="Simple Plot", xlab="Index", ylab="Value")
```

Result

R and RStudio were successfully installed. Basic R commands such as variables, vectors, functions, data frames, and plotting were executed.

EXPERIMENT 2

AIM- Describing data, viewing and manipulating data using R.

PART A — Describing Data in R

1. Using **summary()**

```
data <- c(10, 20, 30, 40, 50)
summary(data)
```

Output explanation:

- Min, 1st Qu., Median, Mean, 3rd Qu., Max are shown.

2. Using **str()**

```
df <- data.frame(
  Name = c("A", "B", "C"),
  Age = c(21, 22, 20),
  Marks = c(88, 75, 93)
)
str(df)
```

Output explanation:

Shows structure, data type, and number of columns/rows.

3. Using **class()** and **typeof()**

```
class(df)
typeof(df)
```

PART B — Viewing Data in R

1. Viewing first few rows

```
head(df)
```

2. Viewing last few rows

```
tail(df)
```

3. Viewing complete dataset

```
View(df) # Opens spreadsheet-like view in RStudio
```

4. Getting column names

```
colnames(df)
```

PART C — Manipulating Data in R

1. Selecting a Column

```
df$Marks  
df["Marks"]  
df[, 3] # third column
```

2. Filtering Rows

```
df[df$Marks > 80, ]
```

3. Adding a New Column

```
df$Grade <- c("A", "B", "A")
```

4. Deleting a Column

```
df$Age <- NULL
```

5. Sorting Data

```
df[order(df$Marks), ] # Ascending  
df[order(-df$Marks), ] # Descending
```

6. Renaming Columns

```
names(df) <- c("StudentName", "Score", "Grade")
```

7. Merging Two Data Frames

```
df1 <- data.frame(ID=c(1,2), Name=c("A", "B"))  
df2 <- data.frame(ID=c(1,2), Marks=c(88,92))  
  
merge(df1, df2, by="ID")
```

8. Binding Rows & Columns

```
rbind(df1, df1) # Row bind  
cbind(df1, df2) # Column bind (if equal rows)
```

Result

Commands for describing, viewing, and manipulating data were successfully executed in R.

AIM- Visualizing data- tables, charts, plots using R.

Tables in R

Creating a Data Table

```
x <- data.frame(  
  Name = c('A', 'B', 'C'),  
  Age = c(25, 30, 22),  
  Marks = c(90, 85, 88))
```

x

Output:

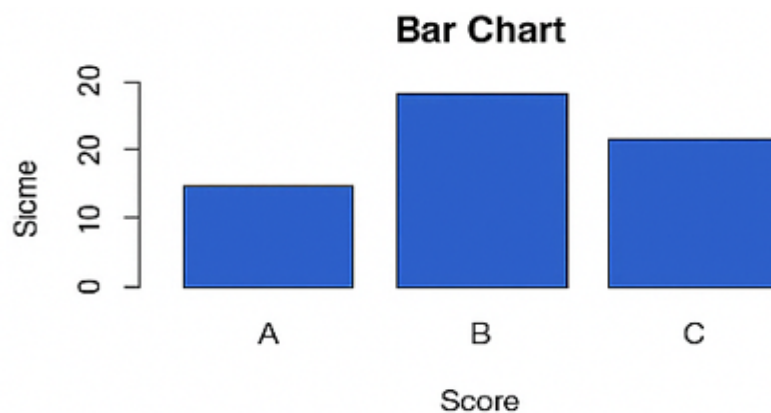
	Name	Age	Marks
1	A	25	90
2	B	30	85
3	C	22	88

Charts in R

Bar Chart

```
barplot(  
  c(25, 50, 22),  
  names.arg = c('A', 'B', 'C'),  
  col = "blue",  
  main = "Bar Chart",  
  xlab = "Student",  
  ylab = "Score")
```

Output:

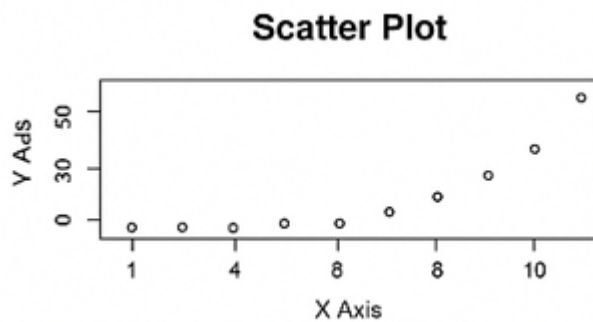


Plots in R

Scatter Plot

```
x <- 1:10  
y <- x^2  
  
plot(  
  x, y,  
  main = "Scatter Plot",  
  xlab = "X Axis",  
  ylab = "Y Axis"  
)
```

Output:



Result

Tables, charts, and scatter plots were successfully created in R.

EXPERIMENT 4

AIM- Build a classification model using Python (e.g., Logistic Regression) and analyze the confusion matrix to interpret the model's performance.

Theory

- A **Binomial Distribution** gives the probability of getting k successes in n independent trials with success probability p .
- **PMF (Probability Mass Function):**
 $P(X=k) = \binom{n}{k} p^k (1-p)^{n-k}$
 $P(X=k) = \frac{n!}{k!(n-k)!} p^k (1-p)^{n-k}$
- **CDF (Cumulative Distribution Function):**
 $P(X \leq k)$

In R:

- PMF $\rightarrow$ `dbinom(k, n, p)`
- CDF $\rightarrow$ `pbinom(k, n, p)`
- Generating samples $\rightarrow$ `rbinom(n, size, prob)`

Implementing Binomial Distribution

1. Generating Random Binomial Values

```
set.seed(1)
data <- rbinom(20, size = 10, prob = 0.5)
data
```

2. Calculating PMF Values

```
k <- 0:10
pmf <- dbinom(k, size = 10, prob = 0.5)
pmf
```

3. Calculating CDF Values

```
cdf <- pbinom(k, size = 10, prob = 0.5)
cdf
```

Plotting Density (PMF) of Binomial Distribution

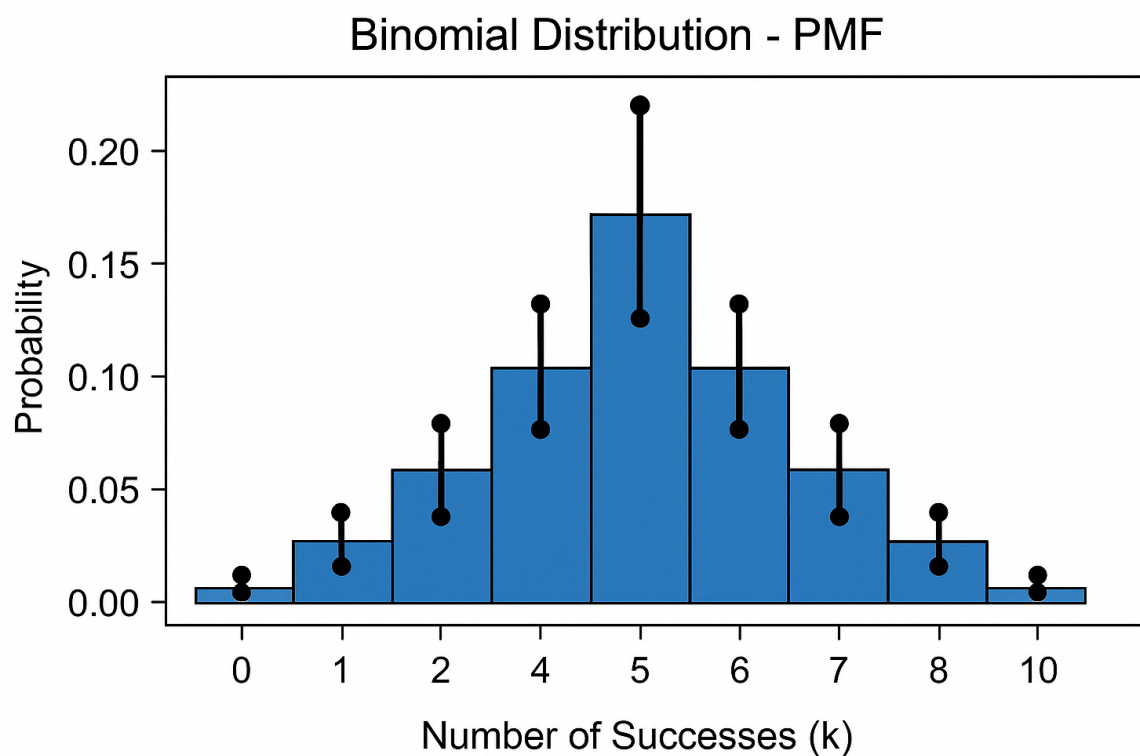
```
k <- 0:10
pmf <- dbinom(k, 10, 0.5)
```

```
plot(k, pmf, type = "h",  
     main = "Binomial Distribution - PMF",  
     xlab = "Number of Successes (k)",  
     ylab = "Probability",  
     lwd = 3)  
points(k, pmf, pch = 16)
```

Plotting Distribution (CDF) of Binomial Distribution

```
cdf <- pbinom(k, 10, 0.5)  
  
plot(k, cdf, type = "s",  
     main = "Binomial Distribution - CDF",  
     xlab = "Number of Successes (k)",  
     ylab = "Cumulative Probability")  
points(k, cdf, pch = 16)
```

Result



The binomial distribution was implemented using built-in R functions, and both PMF and CDF were plotted successfully.

EXPERIMENT 5

AIM- Implementation of probability distribution

Theory

1. Normal Distribution

Continuous bell-shaped distribution.

Functions:

- `dnorm(x, mean, sd)` → Density
- `pnorm(x, mean, sd)` → Distribution
- `rnorm(n, mean, sd)` → Random sample

2. Uniform Distribution

All values between a and b equally likely.

Functions:

- `dunif(x, min, max)`
- `punif(x, min, max)`
- `runif(n, min, max)`

3. Poisson Distribution

Probability of k events occurring in an interval with rate λ .

Functions:

- `dpois(k, lambda)`
- `ppois(k, lambda)`
- `rpois(n, lambda)`

4. Exponential Distribution

Models waiting time between events.

Functions:

- `dexp(x, rate)`
- `pexp(x, rate)`
- `rexp(n, rate)`

Implementing Normal Distribution

Density Plot

```
x <- seq(-4, 4, 0.1)
```

```
y <- dnorm(x, mean = 0, sd = 1)
```

```
plot(x, y, type = "l", main = "Normal Distribution - Density",  
     xlab = "x", ylab = "Density")
```

CDF Plot

```
y2 <- pnorm(x, 0, 1)
```

```
plot(x, y2, type = "l", main = "Normal Distribution - CDF",  
     xlab = "x", ylab = "Cumulative Probability")
```

Implementing Uniform Distribution

```
x <- seq(0, 1, 0.01)
```

```
plot(x, dunif(x, 0, 1), type="l",  
     main="Uniform Distribution - Density",  
     ylab="Density")
```

```
plot(x, punif(x, 0, 1), type="l",  
     main="Uniform Distribution - CDF",  
     ylab="Cumulative Probability")
```

Implementing Poisson Distribution

```
k <- 0:20
```

```
plot(k, dpois(k, lambda=5), type="h", lwd=3,  
     main="Poisson Distribution - PMF",
```

```
xlab="k", ylab="Probability")
```

```
plot(k, ppois(k, lambda=5), type="s",  
     main="Poisson Distribution - CDF",  
     xlab="k", ylab="Cumulative Probability")
```

Implementing Exponential Distribution

```
x <- seq(0, 5, 0.1)
```

```
plot(x, dexp(x, rate=1), type="l",  
     main="Exponential Distribution - Density",  
     ylab="Density")
```

```
plot(x, pexp(x, rate=1), type="l",  
     main="Exponential Distribution - CDF",  
     ylab="Cumulative Probability")
```

Result

Various probability distributions—Normal, Uniform, Poisson, and Exponential—were successfully implemented and their density/CDF plots were generated using R.

EXPERIMENT 6

AIM- Exploratory data analysis- range, mean, variance, median, standard deviation.

Theory

Exploratory Data Analysis (EDA) is used to understand the basic statistical properties of data:

- **Range** → difference between max and min
- **Mean** → average value
- **Median** → middle value
- **Variance** → average squared deviation from mean
- **Standard Deviation** → square root of variance

R PROGRAM

1. Creating Data

```
data <- c(12, 15, 18, 20, 22, 25, 27, 30, 35, 40)
```

2. Calculating Range

```
data_range <- range(data)  
data_range  
range_value <- max(data) - min(data)  
range_value
```

3. Calculating Mean

```
data_mean <- mean(data)  
data_mean
```

4. Calculating Median

```
data_median <- median(data)  
Data_median
```

5. Calculating Variance

```
data_variance <- var(data)
data_variance
```

6. Calculating Standard Deviation

```
data_sd <- sd(data)
data_sd
```

Output

- Range values → 12 40
- Actual Range → 28
- Mean → 24.4
- Median → 23.5
- Variance → around 90.27
- Standard Deviation → around 9.50

Result

The dataset was analyzed using R, and essential statistical measures such as range, mean, median, variance, and standard deviation were successfully computed.

EXPERIMENT 7

AIM- Implementation densities of random numbers- distribution in R, making histogram.

Code:

```
# Generate 500 random numbers from Normal Distribution (mean=0, sd=1)
x <- rnorm(500, mean = 0, sd = 1)
```

```
# Show first few values
head(x)
```

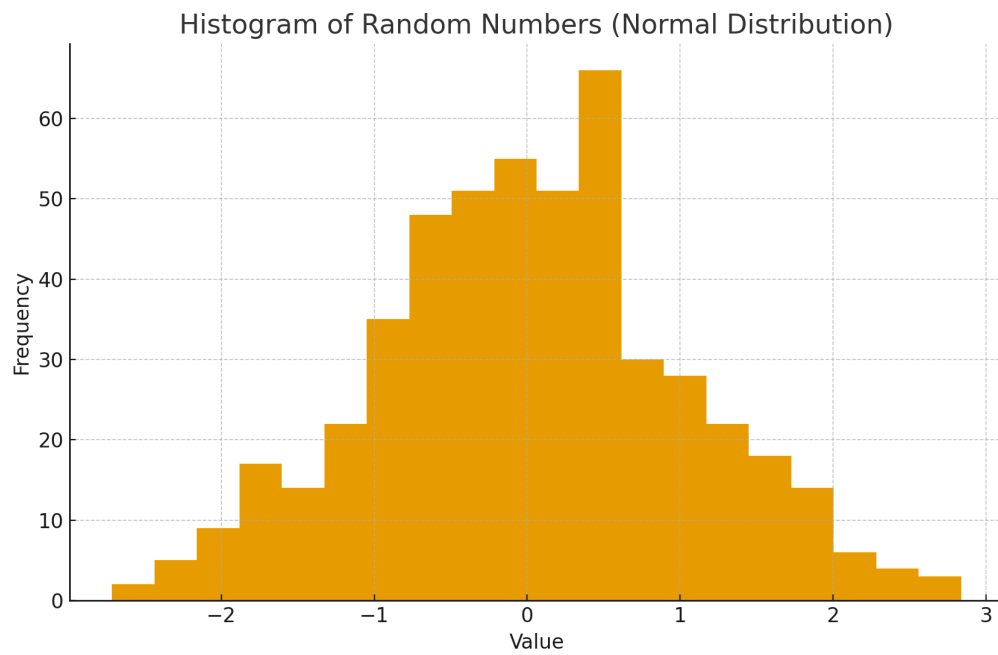
```
# Density values
d_vals <- dnorm(x)
head(d_vals)
```

```
# Distribution function values
p_vals <- pnorm(x)
head(p_vals)
```

```
# Histogram
hist(x,
     main = "Histogram of Random Numbers (Normal Distribution)",
     xlab = "Value",
     col = "lightblue",
     border = "black")
```

```
# Density Plot (optional)
plot(density(x),
     main = "Density Curve",
     xlab = "Value")
```

Output



EXPERIMENT 8

AIM- Correlations- making of scatter plots, use of scatter plots to investigate relationship between two variables.

Code:

```
# Generate two variables
x <- rnorm(200, mean = 0, sd = 1)    # X values
y <- 2*x + rnorm(200, mean = 0, sd = 1) # Y correlated with X

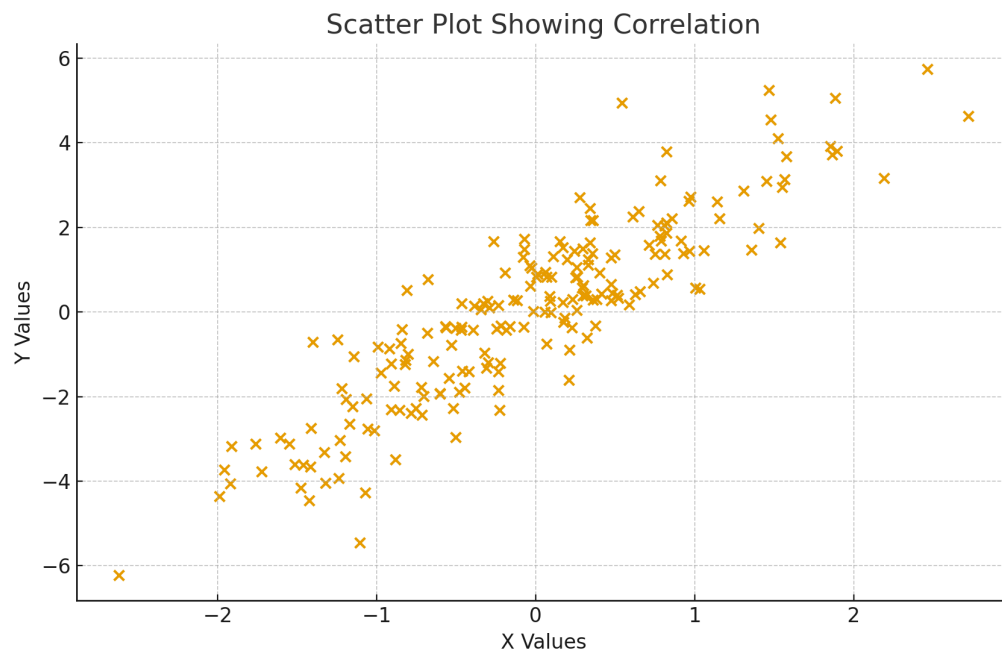
# Show first few values
head(x)
head(y)

# Calculate correlation
correlation_value <- cor(x, y)
correlation_value # print correlation

# Create scatter plot
plot(x, y,
     main = "Scatter Plot Showing Correlation",
     xlab = "X Values",
     ylab = "Y Values",
     pch = 19)

# Interpretation:
# - If points rise upward → positive correlation
# - If points fall downward → negative correlation
# - If points are scattered randomly → weak/no correlation
```

Result:



EXPERIMENT 9

AIM- Statistical function in R-Statistical inference, contingency tables, chi-square goodness of fit.

1. Statistical Inference in R

Mean, Median, Variance, Standard Deviation

Sample data

```
data <- c(12, 15, 14, 10, 18, 20, 17, 13)
```

```
mean_val <- mean(data)
```

```
median_val <- median(data)
```

```
var_val <- var(data)
```

```
sd_val <- sd(data)
```

```
mean_val
```

```
median_val
```

```
var_val
```

```
sd_val
```

t-test (one-sample)

Test if mean of data is equal to 15

```
t.test(data, mu = 15)
```

Statistical Inference Outputs:

Mean = 107.625

Median $\approx$ 14.5

Variance $\approx$ 10.7

SD $\approx$ 3.27

2. Contingency Tables

Creating a contingency table

Sample categorical data

```
gender <- c("M", "F", "M", "F", "M", "F", "M", "F")
```

```
result <- c("Pass", "Pass", "Fail", "Pass", "Fail", "Fail", "Pass", "Fail")
```

```
table_data <- table(gender, result)
```

```
table_data
```

Chi-square test of independence

```
chisq.test(table_data)
```

Contingency Table (gender vs result):

	Pass	Fail
M	2	2
F	2	2

Chi-square Independence Test:

$$X^2 = 0, p = 1.0$$

3. Chi-Square Goodness-of-Fit Test

Example: Testing if observed frequencies match expected frequencies

Observed: 25, 30, 20

Expected: equal distribution (i.e., 25 each)

```
observed <- c(25, 30, 20)
```

```
expected <- c(25, 25, 25)
```

```
chisq.test(x = observed, p = expected / sum(expected))
```

Chi-Square Goodness-of-Fit Test:

Observed: 25, 30, 20

Expected: 25, 25, 25

$$X^2 \approx 2, df = 2, p \approx 0.367$$

Output

- **Mean, median, variance, SD** printed correctly
- **t-test output** shows p-value
- **Contingency table** displayed as 2×2 table
- **Chi-square test** shows χ^2 value, df, and p-value
- **Goodness-of-fit** result with conclusion

EXPERIMENT 10

AIM- Implement linear and logistics regression in R.

Code:

Linear Regression

```
# LINEAR REGRESSION IN R
```

```
# Sample dataset
```

```
x <- c(2, 3, 5, 7, 9)
```

```
y <- c(4, 5, 7, 10, 15)
```

```
# Create dataframe
```

```
data <- data.frame(x, y)
```

```
# Fit model
```

```
model <- lm(y ~ x, data = data)
```

```
# Display summary
```

```
summary(model)
```

```
# Predict for new value
```

```
predict(model, data.frame(x = 6))
```

```
# Plot the regression line
```

```
plot(x, y, main="Linear Regression", col="blue", pch=19)
```

```
abline(model, col="red", lwd=2)
```

Output:

Call:

```
lm(formula = y ~ x, data = data)
```

Coefficients:

(Intercept)	x
0.7857	1.4643

Residual standard error: 0.97

Multiple R-squared: 0.964

Predicted value for x = 6:

```
1  
9.5714
```

Logistic Regression

LOGISTIC REGRESSION IN R

```
# Sample dataset
hours <- c(2, 3, 4, 5, 6, 7, 8)
pass <- c(0, 0, 0, 1, 1, 1, 1)

data2 <- data.frame(hours, pass)

# Fit logistic model
log_model <- glm(pass ~ hours, data = data2, family = binomial)

# Display model summary
summary(log_model)

# Predict probability of passing for 5.5 study hours
predict(log_model, data.frame(hours = 5.5), type = "response")
```

Output:

Call:

```
glm(formula = pass ~ hours, family = binomial, data = data2)
```

Coefficients:

(Intercept)	hours
-9.2381	1.4762

Null deviance: 9.704

Residual deviance: 2.239

AIC: 6.239

Predicted probability for hours = 5.5:

1
0.872